

```

/* Program create Graph structure by "2 dimension array
representation method".
Can show the result of graph are ..
1. Set of vertex
2. Set of Edge
Note.- can use both Undirected graph and Directed graph
*/
#include <stdio.h> //use printf
#include <conio.h> //use getch
#define MaxNode 6 // Define Max Node of Graph
int graph[MaxNode][MaxNode] = {
    {0, 1, 1, 1, 0, 0},
    {1, 0, 1, 0, 1, 0},
    {1, 1, 0, 0, 0, 0},
    {1, 0, 0, 0, 1, 1},
    {0, 1, 0, 1, 0, 0},
    {0, 0, 0, 1, 0, 0}}; //
Declare array and keep data of graph
char NodeName[MaxNode] = {'A', 'B', 'C', 'D', 'E', 'F'}; //
Keep Node Name
void DispArray2D() //
Display value in 2D Array
{
    int i, j; // i=Row, j=Column
    printf(" ");
    for (j = 0; j <= MaxNode; j++) // Display column name of
array
        printf("%c ", NodeName[j]);
    printf("\n"); // line feed
    for (i = 0; i < MaxNode; i++) // row loop
    {
        printf("%c ", NodeName[i]); // Display row name of
array
        for (j = 0; j < MaxNode; j++) // column loop
            printf("%d ", graph[i][j]); // Display value path
        printf("\n");
    }
}
void DispSetOfVertex() // Display set of Vertex
{
    int i;
    printf("\nSet of Vertex = {");

```

```

    for (i = 0; i < MaxNode; i++)
    {
        printf("%c", NodeName[i]); // Display each node name
        if (i != MaxNode - 1)
            printf(",");
    }
    printf("}\n");
}
void DispSetOfEdge() // Display set of Edge
{
    int i, j;
    printf("\nSet of Edge = {");
    for (i = 0; i < MaxNode; i++) // row loop
        for (j = 0; j < MaxNode; j++) // column loop
        {
            if (graph[i][j] == 1)
                printf("(%c,%c),", NodeName[i], NodeName[j]);
        }
    // Show each Edge
    printf("}\n");
}
int main()
{
    printf("GRAPH (ADJACENCY MATRIX REPRESENTATION METHOD)\n");

    printf("=====\n");
    DispArray2D();
    DispSetOfVertex();
    DispSetOfEdge();
    getch();
    return (0);
} // End Main

```

โปรแกรมนี้เป็นการสร้างโครงสร้างข้อมูลแบบกราฟ (Graph) โดยใช้วิธี Adjacency Matrix Representation หรือการแทนกราฟด้วยอาร์เรย์สองมิติ (2D Array) ซึ่งเป็นวิธีที่นิยมใช้ในการเก็บข้อมูลความสัมพันธ์ระหว่างจุดยอด (Vertex) ของกราฟ แต่ละช่องในเมทริกซ์จะเก็บค่า 0 หรือ 1 เพื่อระบุว่าจุดยอดสองจุดมีเส้นเชื่อม (Edge) ระหว่างกันหรือไม่ ในโปรแกรมนี้กำหนดจุดยอดทั้งหมด 6 จุด ได้แก่ A, B, C, D, E และ F และกำหนดความสัมพันธ์ระหว่างจุดยอดเหล่านี้ไว้ภายในตัวแปร graph

ซึ่งอยู่ในรูปของเมทริกซ์ขนาด 6×6 เมื่อโปรแกรมเริ่มทำงาน จะทำการแสดงหัวข้อของโปรแกรม จากนั้นเรียกใช้ฟังก์ชัน `DispArray2D()` เพื่อแสดงข้อมูลภายใน Adjacency Matrix โดยแสดงชื่อของจุดยอดทั้งในแนวแถวและแนวคอลัมน์

ทำให้สามารถมองเห็นได้ว่าจุดยอดใดเชื่อมต่อกับจุดยอดใด ตัวอย่างเช่น ค่า `graph[0][1] = 1`

หมายความว่า Vertex A มีการเชื่อมต่อกับ Vertex B ส่วนค่า 0

หมายถึงไม่มีการเชื่อมต่อระหว่างจุดยอดทั้งสอง หลังจากแสดงเมทริกซ์แล้ว

โปรแกรมจะเรียกใช้ฟังก์ชัน `DispSetOfVertex()` เพื่อแสดงเซตของจุดยอดทั้งหมดในรูปแบบ

{A,B,C,D,E,F} ซึ่งเป็นการแสดงสมาชิกทั้งหมดของกราฟ ทำให้ทราบว่ากราฟนี้ประกอบด้วย

Vertex จำนวน 6 จุด ถัดมาจะเรียกใช้ฟังก์ชัน `DispSetOfEdge()` เพื่อแสดงเซตของเส้นเชื่อม

โดยโปรแกรมจะตรวจสอบทุกตำแหน่งในเมทริกซ์ หากพบค่าเป็น 1 จะแสดงคู่ลำดับของจุดยอด

เช่น (A,B) หรือ (D,F) ซึ่งหมายถึงมีเส้นเชื่อมระหว่างจุดยอดคู่ นั้น

ผลลัพธ์ที่ได้จะแสดงเส้นเชื่อมทั้งหมดที่ปรากฏอยู่ในเมทริกซ์

จากผลลัพธ์ที่แสดงสามารถสรุปได้ว่ากราฟนี้ประกอบด้วย Vertex จำนวน 6 จุด

และมีความสัมพันธ์ระหว่างจุดยอดหลายคู่ เช่น A เชื่อมกับ B, C และ D ส่วน B เชื่อมกับ A, C และ

E เป็นต้น โดยสังเกตได้ว่าค่าในเมทริกซ์มีลักษณะสมมาตรกัน เช่น (A,B) และ (B,A) มีค่าเท่ากัน

แสดงว่ากราฟตัวอย่างนี้เป็น Undirected Graph หรือกราฟไม่มีทิศทาง

ในระหว่างการพัฒนาโปรแกรมพบประเด็นที่ควรปรับปรุงอย่างหนึ่งคือ ในฟังก์ชัน `DispArray2D()`

มีคำสั่ง

```
for (j = 0; j <= MaxNode; j++)
```

ซึ่งทำให้ลูปทำงานถึงค่า $j = 6$ ทั้งที่อาร์เรย์ `NodeName` มีสมาชิกตั้งแต่ตำแหน่ง 0 ถึง 5 เท่านั้น

อาจทำให้โปรแกรมอ่านข้อมูลเกินขอบเขตอาร์เรย์และแสดงค่าที่ไม่ถูกต้องได้

วิธีแก้ไขคือเปลี่ยนเงื่อนไขเป็น

```
for (j = 0; j < MaxNode; j++)
```

นอกจากนี้ ในการแสดงเซตของ Edge โปรแกรมจะแสดงคู่ข้อมูลซ้ำกัน เช่น (A,B) และ (B,A)

เนื่องจากกราฟเป็นแบบไม่มีทิศทาง ซึ่งจริง ๆ แล้วถือเป็นเส้นเชื่อมเดียวกัน

วิธีแก้ไขคือให้ตรวจสอบเฉพาะครึ่งบนหรือครึ่งล่างของเมทริกซ์เพียงด้านเดียว

เพื่อลดการแสดงผลซ้ำซ้อน โดยสรุป โปรแกรมนี้สามารถแสดงแนวคิดการจัดเก็บข้อมูลกราฟด้วย

Adjacency Matrix ได้อย่างชัดเจน ช่วยให้เข้าใจความสัมพันธ์ระหว่าง Vertex และ Edge

ภายในกราฟ สามารถแสดงเมทริกซ์ เซตของจุดยอด และเซตของเส้นเชื่อมได้ครบถ้วน

เหมาะสำหรับการศึกษาโครงสร้างข้อมูลกราฟเบื้องต้น

และสามารถนำไปพัฒนาต่อยอดสำหรับการค้นหาเส้นทางหรืออัลกอริทึมที่เกี่ยวข้องกับกราฟในอนาคตได้.

```

/* Program create Graph structure by "Sparse Matrix
representation method".
Can show the result of graph are ..
1. Set of vertex
2. Set of Edge
Note.- can use both Undirected graph and Directed graph
*/

#include <stdio.h> //use printf
#include <conio.h> //use getch
#include <stdlib.h> //use malloc
#define MaxNode 4 // Define Max Node of Graph
char NodeName[4] = {'A', 'B', 'C', 'D'};
int graph[MaxNode][MaxNode] = {
    {0, 1, 0, 0},
    {1, 0, 1, 1},
    {1, 0, 0, 1},
    {0, 0, 0, 0},
}; // Declare array and keep data of graph
typedef struct Node // Declare structure of every node
{
    char V1, V2;
    struct Node *Row, *Column;
} Node;
struct Node *Start[MaxNode], *p, *Head; // Declare pointer
uses
Node *Allocate() // Allocate 1 node
from storage pool
{
    struct Node *temp;
    temp = (Node *)malloc(sizeof(Node)); // Allocate node by
size declare
    return (temp);
}
void CreateHeadRC() // Create Head Node
{
    int i, j;
    struct Node *row, *col;
    p = Allocate(); // Create HEAD Node
    Head = p; // Let Head point at this Node and FREEZE
it
    row = Head;

```

```

    for (i = 0; i < MaxNode; i++) // Create HEAD ROW
    {
        p = Allocate();
        row->Row = p;          // Let ROW OF TEMP point to new
node
        row = p;              // Change temp point to new
node
        row->V1 = NodeName[i]; // Set Node Name
        row->Row = NULL;
        row->Column = NULL;
    }
    col = Head;
    for (j = 0; j < MaxNode; j++) // Create HEAD COLUMN
    {
        p = Allocate();
        col->Column = p;          // Let COLUMN OF TEMP point to
new node
        col = p;                // Change temp point to new
node
        col->V2 = NodeName[j]; // Set Node Name
        col->Row = NULL;
        col->Column = NULL;
    }
}

void TransferToSparseMT() // Transfer array to Sparse Matrix
of graph
{
    int i, j;
    struct Node *row1, *row2, *col1, *col2;
    row1 = Head->Row;
    i = 0;          // Start of Row in Array
    while (row1 != NULL) // row loop
    {
        col1 = Head->Column;
        j = 0;          // Start of Column in Array
        col2 = row1;     // Let COL2 Count Column2 at ROW
Pointed
        while (col1 != NULL) // column loop
        {
            if (graph[i][j] == 1) // if PATH?
            {
                p = Allocate();          // get new Node

```

```

        p->V1 = row1->V1;          // Set arrow tail of
Edge
        p->V2 = col1->V2;          // Set head tail of
Edge
        p->Row = NULL;             // Set pointer as
NULL default
        p->Column = NULL;          // Set pointer as
NULL default
        col2->Column = p;          // Let Column2 at
ROW point to New Node
        col2 = p;                  // Change COL2
position
        row2 = col1;              // Mark ROW2 point
at first ROW of COLUMN
        while (row2->Row != NULL) // Find the last
Node
            row2 = row2->Row;      // Skip to next Node
            row2->Row = p;          // Let Row of Column
point to this node
        }
        col1 = col1->Column;
        j++; // Skip to next Column Edge of Array
    }
    row1 = row1->Row;
    i++; // Skip to next Row Edge of Array
}
}
void DispSetOfVertex() // Display set of Vertex
{
    struct Node *row;
    printf("\nSet of Vertex = {");
    row = Head->Row;    // Start Row at here
    while (row != NULL) // Loop at Row direction
    {
        printf("%c", row->V1); // Display each node name
        if (row->Row != NULL)
            printf(",");
        row = row->Row; // Skip to next Row
    }
    printf("}\n");
}
void DispSetOfEdge() // Display set of Edge

```

```

{
    struct Node *row, *col;
    printf("\nSet of Edge = {");
    row = Head->Row;    // Start point of Row
    while (row != NULL) // row loop
    {
        col = row->Column; // Start point of Column
        while (col != NULL) // Column loop
        {
            printf("(%c,%c),", col->V1, col->V2); // Show each
Edge
            col = col->Column; // Skip to
next Edge/Column
        }
        row = row->Row; // Skip to next Row
    }
    printf("}\n");
}
int main()
{
    printf("GRAPH SPARSE MATRIX REPRESENTATION METHOD\n");
    printf("=====\n");
    CreateHeadRC();
    TransferToSparseMT();
    DispSetOfVertex();
    DispSetOfEdge();
    getch();
    return (0);
} // End Main

```

โปรแกรมนี้เป็นการสร้างโครงสร้างข้อมูลกราฟ (Graph) โดยใช้วิธี Sparse Matrix Representation ซึ่งเป็นการเก็บข้อมูลเฉพาะตำแหน่งที่มีการเชื่อมต่อกันจริง แทนที่จะเก็บข้อมูลครบทุกตำแหน่งเหมือน Adjacency Matrix วิธีนี้ช่วยประหยัดหน่วยความจำ โดยเฉพาะกราฟที่มีจำนวน Vertex มาก แต่มีจำนวน Edge น้อย โปรแกรมกำหนดจุดยอดทั้งหมด 4 จุด ได้แก่ A, B, C และ D และเก็บข้อมูลความเชื่อมโยงเริ่มต้นไว้ในอาร์เรย์สองมิติ จากนั้นจะนำข้อมูลดังกล่าวมาแปลงเป็นโครงสร้างแบบลิงก์ลิสต์ (Linked Structure) ที่สามารถเชื่อมโยงข้อมูลระหว่างแถวและคอลัมน์ได้ การทำงานของโปรแกรมเริ่มจากการสร้าง Head Node และสร้างโหนดหัวแถว (Row Header) และหัวคอลัมน์ (Column Header) สำหรับแต่ละ

Vertex ผ่านฟังก์ชัน CreateHeadRC() เพื่อใช้เป็นจุดเริ่มต้นในการเชื่อมโยงข้อมูลทั้งหมด หลังจากนั้นฟังก์ชัน TransferToSparseMT() จะทำการตรวจสอบข้อมูลใน Adjacency Matrix ที่ละตำแหน่ง หากพบค่าเป็น 1 ซึ่งหมายถึงมีเส้นเชื่อมระหว่าง Vertex สองจุด โปรแกรมจะสร้างโหนดใหม่ขึ้นมาเพื่อเก็บข้อมูลของ Edge นั้น พร้อมเชื่อมโยงเข้ากับรายการของแถวและคอลัมน์ที่เกี่ยวข้อง ทำให้สามารถเข้าถึงข้อมูลได้ทั้งตามแนวแถวและแนวคอลัมน์ เมื่อสร้าง Sparse Matrix เสร็จแล้ว โปรแกรมจะเรียกใช้ฟังก์ชัน DispSetOfVertex() เพื่อแสดงชุดของ Vertex ทั้งหมด ซึ่งผลลัพธ์ที่ได้คือ

Set of Vertex = {A,B,C,D}

จากนั้นเรียกใช้ฟังก์ชัน DispSetOfEdge() เพื่อแสดงชุดของ Edge ที่ถูกสร้างขึ้น โดยตรวจสอบโหนดข้อมูลที่เชื่อมต่ออยู่ในแต่ละแถวและแสดงผลเป็นคู่ลำดับของจุดยอด ตัวอย่างผลลัพธ์ที่ได้คือ

Set of Edge = { 2 (A,B), 3 (B,A), 4 (B,C), 5 (B,D), 6 (C,A), 7 (C,D) 8 }

ผลลัพธ์ดังกล่าวแสดงให้เห็นความสัมพันธ์ระหว่างจุดยอดในกราฟ

โดยแต่ละคู่ลำดับหมายถึงการเชื่อมต่อจากจุดยอดต้นทางไปยังจุดยอดปลายทาง เช่น (A,B) หมายถึง A เชื่อมต่อไปยัง B และ (B,C) หมายถึง B เชื่อมต่อไปยัง C

จากข้อมูลที่กำหนดไว้จะเห็นว่าบางเส้นเชื่อมไม่ได้มีค่าที่สมมาตรกัน เช่น (C,A) มีอยู่ แต่ (A,C) ไม่มีอยู่ในเมทริกซ์ จึงสามารถพิจารณาได้ว่าข้อมูลกราฟตัวอย่างนี้มีลักษณะใกล้เคียงกับ Directed Graph (กราฟมีทิศทาง) มากกว่ากราฟไม่มีทิศทาง เพราะการเชื่อมต่อบางคู่เกิดขึ้นเพียงทิศทางเดียว ในระหว่างการพัฒนาโปรแกรม มีความท้าทายหลักอยู่ที่การจัดการ Pointer

และการเชื่อมโยงโหนดต่าง ๆ ของ Sparse Matrix

เนื่องจากแต่ละโหนดต้องเชื่อมต่อทั้งในแนวแถวและแนวคอลัมน์ หากกำหนด Pointer

ผิดพลาดอาจทำให้ข้อมูลสูญหายหรือเกิดการเข้าถึงหน่วยความจำที่ไม่ถูกต้องได้

การแก้ปัญหาคือกำหนดค่า Row และ Column ของโหนดใหม่ให้เป็น NULL ก่อนทุกครั้ง

และตรวจสอบปลายทางของลิงก์ก่อนเชื่อมต่อโหนดใหม่เข้าไป นอกจากนี้โปรแกรมยังใช้ฟังก์ชัน malloc() เพื่อจองหน่วยความจำแบบไดนามิก

ดังนั้นหากมีการพัฒนาต่อควรเพิ่มการคืนหน่วยความจำด้วย free() เพื่อป้องกันปัญหา Memory

Leak โดยสรุป โปรแกรมนี้เป็นตัวอย่างการแทนข้อมูลกราฟด้วยวิธี Sparse Matrix

ซึ่งมีประสิทธิภาพด้านการใช้หน่วยความจำมากกว่า Adjacency Matrix ในกรณีที่จำนวน Edge

มีน้อยเมื่อเทียบกับจำนวน Vertex โปรแกรมสามารถสร้างโครงสร้างกราฟ แสดงชุดของจุดยอด

และแสดงชุดของเส้นเชื่อมได้อย่างถูกต้อง

อีกทั้งยังช่วยให้เข้าใจหลักการทำงานของโครงสร้างข้อมูลแบบลิงก์ลิสต์และการประยุกต์ใช้ Pointer ในการจัดเก็บข้อมูลกราฟได้อย่างชัดเจน.

```
/* Program create Graph structure by "Adjacency Multi list
method".
Can show the result of graph are ..
1. Set of vertex
2. Set of Edge
*/
#include <stdio.h> //use printf
#include <conio.h> //use getch
#define MaxEdge 8 // Define Max Edge of Graph
#define Block 5 // Define Block of each node
#define MaxNode 5 // Define Max Node
char NodeName[5] = {'A', 'B', 'C', 'D', 'E'};
char graph[MaxEdge][Block] = {
    {'0', 'A', 'B', '2', '5'},
    {'0', 'A', 'C', '3', '5'},
    {'0', 'A', 'D', '4', '6'},
    {'0', 'A', 'E', '-', '7'},
    {'0', 'B', 'C', '6', '7'},
    {'0', 'B', 'D', '-', '8'},
    {'0', 'C', 'E', '-', '8'},
    {'0', 'D', 'E', '-', '-'}
}; // Declare array and keep data of graph
void DispArray2D()
{
    // Display value in 2D Array
    int i, j; // i=Row, j=Column
    printf(" M V1 V2 E1 E2\n");
    for (i = 0; i < MaxEdge; i++)
    {
        // row loop
        printf("%d ", i + 1); // Display number of
Row
        for (j = 0; j < Block; j++) // column loop
            printf("%c ", graph[i][j]); // Display value path
        printf("\n");
    }
}
void DispSetOfVertex()
{ // Display set of Vertex
    int i;
```

```

printf("\nSet of Vertex = {");
for (i = 0; i < MaxNode; i++)
{
    printf("%c", NodeName[i]); // Display each node name
    if (i != MaxNode - 1)
        printf(",");
}
printf("}\n");
}
void DispSetOfEdge()
{ // Display set of Edge
    int i, j;
    printf("\nSet of Edge = {");
    for (i = 0; i < MaxEdge; i++)
    {
        // row
        loop
        printf("(%c,%c),", graph[i][1], graph[i][2]); // Show
        each Edge1
        printf("(%c,%c),", graph[i][2], graph[i][1]); // Show
        each Edge2
    }
    printf("}\n");
}
int main()
{
    printf("GRAPH ADJACENCY MULTI-LIST REPRESENTATION
METHOD\n");
    printf("=====\n");
    DispArray2D();
    DispSetOfVertex();
    DispSetOfEdge();
    getch();
    return (0);
} // End Main

```

โปรแกรมนี้เป็นการสร้างโครงสร้างข้อมูลกราฟด้วยวิธี Adjacency Multi-List Representation ซึ่งเป็นรูปแบบการจัดเก็บข้อมูลกราฟที่เหมาะสมกับกราฟแบบไม่มีทิศทาง (Undirected Graph) โดยเก็บข้อมูลของเส้นเชื่อม (Edge) เพียงครั้งเดียว แต่สามารถอ้างอิงถึงจุดยอด (Vertex) ที่เชื่อมต่อกันได้ทั้งสองด้าน วิธีนี้ช่วยลดความซ้ำซ้อนของข้อมูลเมื่อเทียบกับ Adjacency Matrix

และทำให้สามารถจัดการข้อมูลกราฟได้มีประสิทธิภาพมากขึ้น

ภายในโปรแกรมกำหนดจุดยอดทั้งหมด 5 จุด ได้แก่ A, B, C, D และ E

ส่วนข้อมูลเส้นเชื่อมถูกจัดเก็บไว้ในอาร์เรย์สองมิติชื่อ graph ซึ่งแต่ละแถวแทน Edge หนึ่งเส้น

โดยแต่ละ Edge จะประกอบด้วยข้อมูล 5 ส่วน ได้แก่ สถานะ (M), จุดยอดต้นทาง (V1),

จุดยอดปลายทาง (V2), Edge ถัดไปของ V1 (E1) และ Edge ถัดไปของ V2 (E2)

แนวคิดนี้จำลองโครงสร้างของ Adjacency Multi-List ที่แต่ละ Edge สามารถเชื่อมโยงไปยัง Edge

อื่น ๆ ที่เกี่ยวข้องกับ Vertex เดียวกันได้ เมื่อโปรแกรมเริ่มทำงาน จะเรียกฟังก์ชัน DispArray2D()

เพื่อแสดงข้อมูลทั้งหมดในโครงสร้างอาร์เรย์

โดยข้อมูลจะถูกแสดงออกมาเป็นตารางที่ประกอบด้วยหมายเลข Edge และค่าของแต่ละฟิลด์

ทำให้สามารถมองเห็นโครงสร้างการเชื่อมโยงระหว่าง Vertex ต่าง ๆ ได้ ตัวอย่างเช่น แถวแรก

0 A B 2 5

หมายความว่า Edge นี้เชื่อมระหว่าง A และ B โดย Edge ถัดไปที่เกี่ยวข้องกับ A คือหมายเลข 2 และ

Edge ถัดไปที่เกี่ยวข้องกับ B คือหมายเลข 5 หลังจากนั้นโปรแกรมจะเรียกใช้ฟังก์ชัน

DispSetOfVertex() เพื่อแสดงเซตของจุดยอดทั้งหมดในกราฟ ซึ่งผลลัพธ์ที่ได้คือ

Set of Vertex = {A,B,C,D,E}

แสดงว่าในกราฟนี้มี Vertex ทั้งหมด 5 จุด ต่อมาโปรแกรมจะเรียกใช้ฟังก์ชัน DispSetOfEdge()

เพื่อแสดงเซตของเส้นเชื่อมทั้งหมด โดยในแต่ละแถวของข้อมูล Edge

โปรแกรมจะแสดงความสัมพันธ์สองทิศทาง เช่น

(A,B) (B,A)

เนื่องจากกราฟประเภทนี้เป็นกราฟไม่มีทิศทาง การเชื่อมต่อระหว่าง A และ B

สามารถเดินทางได้ทั้งสองทาง จึงมีการแสดงผลทั้ง (A,B) และ (B,A) เช่นเดียวกับ Edge อื่น ๆ

ในกราฟ ผลลัพธ์ที่ได้จะประกอบด้วยลำดับของ Vertex ที่เชื่อมต่อกันทั้งหมด

จากข้อมูลที่กำหนดไว้สามารถสรุปได้ว่ากราฟนี้มี Edge ทั้งหมด 8 เส้น ได้แก่ A-B, A-C, A-D, A-

E, B-C, B-D, C-E และ D-E ซึ่งแสดงให้เห็นว่า Vertex A เป็นจุดยอดที่มีการเชื่อมต่อกับ Vertex

อื่นมากที่สุดในการพัฒนาโปรแกรม มีข้อสังเกตว่าการแสดงผลในฟังก์ชัน

DispSetOfEdge() มีการแสดงข้อมูลซ้ำ เช่น (A,B) และ (B,A)

แม้ว่าจะมีวัตถุประสงค์เพื่อแสดงลักษณะของกราฟแบบไม่มีทิศทาง แต่หากต้องการแสดงเซตของ

Edge ตามหลักคณิตศาสตร์จริง ๆ ควรแสดงเพียงครั้งเดียว เช่น

{(A,B),(A,C),(A,D),(A,E),(B,C),(B,D),(C,E),(D,E)}

เพราะคู่ลำดับทั้งสองถือเป็น Edge เดียวกัน

นอกจากนี้โปรแกรมยังใช้เพียงอาร์เรย์สองมิติในการจำลอง Adjacency Multi-List เท่านั้น

ยังไม่ได้สร้างโครงสร้างแบบลิงก์ลิสต์หรือ Pointer เชื่อมโยงกันจริง ๆ

ดังนั้นจึงถือเป็นการจำลองแนวคิดของ Adjacency Multi-List

เพื่อให้เข้าใจหลักการทำงานมากกว่าการนำไปใช้งานในกราฟขนาดใหญ่ โดยสรุป

โปรแกรมนี้มีวัตถุประสงค์เพื่อศึกษาการแทนข้อมูลกราฟด้วยวิธี Adjacency Multi-List

ผ่านการจัดเก็บข้อมูลในรูปของอาร์เรย์สองมิติ สามารถแสดงชุดของ Vertex และ Edge

ได้อย่างชัดเจน

ช่วยให้เห็นแนวคิดการเชื่อมโยงระหว่างเส้นเชื่อมและจุดยอดในกราฟแบบไม่มีทิศทาง

และเป็นพื้นฐานสำคัญสำหรับการศึกษาโครงสร้างข้อมูลกราฟและอัลกอริทึมที่เกี่ยวข้องในระดับที่สูงขึ้นต่อไปได้อย่างดี.

```
/* Program create Graph structure by "Node directory method".
Can show the result of graph are ..
1. Set of vertex
2. Set of Edge and Weight
Note.- Can use for Weighting Graph in both Undirected graph
and
Directed graph
*/
#include <stdio.h> //use printf
#include <conio.h> //use getch
#define MaxNode 4 // Define Max Node
#define Block 4 // Define Block of each node
#define MaxEdge 6 // Define Max Extra of Graph
char Head[MaxNode][Block] = {
    {'A', '-', '1', '1'},
    {'B', '-', '3', '2'},
    {'C', '-', '2', '5'},
    {'D', '-', '0', '\0'}, // {'D', '-', '0', NULL},
}; // Declare array and keep Head Node of graph
char Edge[MaxEdge][2] = {
    {'2', 'B'},
    {'5', 'A'},
    {'9', 'C'},
    {'7', 'D'},
    {'8', 'B'},
    {'6', 'D'},
}; // Declare array and keep Edge of graph
void DispHead() // Display Head in 2D Array
{
    int i, j; // i=Row, j=Column
```

```

printf("NODE...\n");
printf("No. Name Data Edge Pointer\n");
for (i = 0; i < MaxNode; i++) // row loop
{
    printf("%d ", i + 1);          // Display number of
Row
    for (j = 0; j < Block; j++)    // column loop
        printf("%c ", Head[i][j]); // Display Node
    printf("\n");
}
}

void DispEdge() // Display Edge in 2D Array
{
    int i, j; // i=Row, j=Column
    printf("EDGE...\n");
    printf("No. Weight Node\n");
    for (i = 0; i < MaxEdge; i++) // row loop
    {
        printf("%d ", i + 1);          // Display number of
Row
        for (j = 0; j < 2; j++)          // column loop
            printf("%c ", Edge[i][j]); // Display Node
        printf("\n");
    }
}

void DispSetOfVertex() // Display set of Vertex
{
    int i;
    printf("\nSet of Vertex = {");
    for (i = 0; i < MaxNode; i++)
    {
        printf("%c", Head[i][0]); // Display each node name
        if (i != MaxNode - 1)
            printf(",");
    }
    printf("}\n");
}

void DispSetOfEdge() // Display set of Edge
{
    int i, j, AmtEdge, PT;
    printf("\nSet of Edge = {");
    for (i = 0; i < MaxNode; i++) // row loop

```

```

{
    AmtEdge = Head[i][2] - 48; // Convert Character to
Integer for Amount ofEdge
    PT = Head[i][3] - 48;      // Convert Character
to Integer for Start point of Array
    for (j = 0; j < AmtEdge; j++) // Loop follow by Amount
of Edge
    {
        printf("(%c%c)%c,", Head[i][0], Edge[PT - 1 +
j][1], Edge[PT - 1 + j][0]);
        // Show Edge and Weight
    }
    printf("}\n");
}
int main()
{
    printf("GRAPH NODE DIRECTORY REPRESENTATION METHOD\n");
    printf("=====\n");
    DispHead();
    DispEdge();
    DispSetOfVertex();
    DispSetOfEdge();
    getch();
    return (0);
} // End Main

```

โปรแกรมนี้เป็นการสร้างโครงสร้างข้อมูลกราฟด้วยวิธี Node Directory Representation ซึ่งเป็นการจัดเก็บข้อมูลกราฟโดยแยกข้อมูลของจุดยอด (Vertex) และเส้นเชื่อม (Edge) ออกจากกัน ทำให้สามารถรองรับกราฟที่มีน้ำหนัก (Weighted Graph) ได้ทั้งแบบมีทิศทาง (Directed Graph) และไม่มีทิศทาง (Undirected Graph)

แนวคิดของวิธีนี้คือเก็บข้อมูลหลักของแต่ละจุดยอดไว้ในตาราง Head และเก็บรายละเอียดของเส้นเชื่อมไว้ในตาราง Edge จากนั้นใช้ตัวชี้ (Pointer) และจำนวนเส้นเชื่อมของแต่ละจุดยอดในการเข้าถึงข้อมูลที่เกี่ยวข้อง โปรแกรมกำหนด Vertex ทั้งหมด 4 จุด ได้แก่ A, B, C และ D โดยข้อมูลของแต่ละ Vertex ถูกเก็บไว้ในอาร์เรย์ Head ซึ่งแต่ละแถวประกอบด้วยชื่อจุดยอด จำนวนเส้นเชื่อมที่เกี่ยวข้อง และตำแหน่งเริ่มต้นของข้อมูลในตาราง Edge ส่วนข้อมูลเส้นเชื่อมถูกเก็บในอาร์เรย์ Edge ซึ่งประกอบด้วยค่าน้ำหนัก (Weight) และชื่อจุดยอดปลายทางที่เชื่อมต่อ ตัวอย่างเช่น ข้อมูลของ

Vertex A ระบุว่าเส้นเชื่อมจำนวน 1 เส้น และเริ่มอ่านข้อมูลจากตำแหน่งที่ 1 ของตาราง Edge ทำให้สามารถทราบได้ว่า A เชื่อมต่อไปยัง B ด้วยน้ำหนัก 2

การทำงานของโปรแกรมเริ่มจากการแสดงข้อมูลในตาราง Head ผ่านฟังก์ชัน DispHead() เพื่อให้เห็นโครงสร้างของ Node Directory ที่เก็บรายละเอียดของแต่ละ Vertex จากนั้นฟังก์ชัน DispEdge() จะแสดงข้อมูลทั้งหมดในตาราง Edge

ซึ่งประกอบด้วยค่าน้ำหนักและจุดยอดปลายทางของแต่ละเส้นเชื่อม

ทำให้ผู้ใช้งานสามารถตรวจสอบข้อมูลกราฟที่ถูกจัดเก็บไว้ได้อย่างชัดเจน

หลังจากนั้นโปรแกรมจะเรียกใช้ฟังก์ชัน DispSetOfVertex()

เพื่อแสดงเซตของจุดยอดทั้งหมดในกราฟ ผลลัพธ์ที่ได้คือ

$$\text{Set of Vertex} = \{A, B, C, D\}$$

ซึ่งแสดงว่ากราฟนี้ประกอบด้วย Vertex ทั้งหมด 4 จุด ถัดมาคือฟังก์ชัน DispSetOfEdge()

ซึ่งเป็นส่วนสำคัญของโปรแกรม

โดยจะอ่านจำนวนเส้นเชื่อมและตำแหน่งเริ่มต้นจากข้อมูลในตาราง Head

แล้วนำไปค้นหาข้อมูลในตาราง Edge เพื่อสร้างชุดของเส้นเชื่อมพร้อมน้ำหนัก

ตัวอย่างผลลัพธ์ที่ได้คือ

$$\text{Set of Edge} = \{(AB)2, (BA)5, (BC)9, (BD)7, (CB)8, (CD)6\}$$

ความหมายของผลลัพธ์คือ (AB)2 หมายถึง A เชื่อมไปยัง B ด้วยน้ำหนัก 2 (BA)5 หมายถึง B เชื่อมไปยัง A ด้วยน้ำหนัก 5 (BC)9 หมายถึง B เชื่อมไปยัง C ด้วยน้ำหนัก 9 (BD)7 หมายถึง B เชื่อมไปยัง D ด้วยน้ำหนัก 7 (CB)8 หมายถึง C เชื่อมไปยัง B ด้วยน้ำหนัก 8 (CD)6 หมายถึง C เชื่อมไปยัง D ด้วยน้ำหนัก 6 จากผลลัพธ์จะเห็นว่ากราฟนี้เป็น Weighted Graph

เนื่องจากทุกเส้นเชื่อมมีการกำหนดค่าน้ำหนักไว้ และยังสามารถมองเป็น Directed Graph ได้ เพราะน้ำหนักของเส้นเชื่อมจาก A ไป B และจาก B ไป A มีค่าไม่เท่ากัน

รวมถึงมีการเก็บข้อมูลแยกเป็นแต่ละ Edge ทำให้ทิศทางของการเชื่อมต่อมีความสำคัญ

ในระหว่างการพัฒนาโปรแกรม

จุดที่ต้องระมัดระวังคือการแปลงข้อมูลตัวเลขที่ถูกเก็บในรูปแบบอักขระ (Character)

ให้เป็นจำนวนเต็ม โดยใช้คำสั่ง

$$\text{AmtEdge} = \text{Head}[i][2] - 48;$$

$$\text{PT} = \text{Head}[i][3] - 48;$$

เนื่องจากข้อมูลถูกเก็บเป็นตัวอักษร เช่น '1', '2', '3' จึงต้องนำค่า ASCII ของเลข 0 (48)

มาลบออกเพื่อให้ได้ค่าตัวเลขจริง

หากไม่แปลงค่าก่อนจะไม่สามารถนำไปคำนวณหรือใช้เป็นตำแหน่งอ้างอิงในอาร์เรย์ได้อย่างถูกต้อง

นอกจากนี้ยังต้องตรวจสอบตำแหน่งเริ่มต้นของข้อมูลในตาราง Edge

ให้สอดคล้องกับจำนวนเส้นเชื่อมของแต่ละ Vertex

มีฉะนั้นอาจเกิดการเข้าถึงข้อมูลผิดตำแหน่งหรือแสดงผลผิดพลาดเคลื่อนได้ โดยสรุป

โปรแกรมนี้แสดงแนวคิดการจัดเก็บข้อมูลกราฟแบบ Node Directory

ซึ่งเป็นอีกวิธีหนึ่งที่เหมาะสำหรับกราฟที่มีน้ำหนัก เพราะแยกการจัดเก็บข้อมูล Vertex และ Edge

ออกจากกันอย่างชัดเจน ช่วยให้การค้นหาและจัดการข้อมูลเส้นเชื่อมทำได้สะดวก

สามารถแสดงทั้งเซตของจุดยอดและเซตของเส้นเชื่อมพร้อมน้ำหนักได้อย่างถูกต้อง

และเป็นพื้นฐานสำคัญสำหรับการศึกษากราฟแบบ Weighted Graph

และอัลกอริทึมด้านเส้นทางสั้นที่สุดในลำดับถัดไป.